PROYECTO FINAL

Documentación Completa de la API de PetCare

PetCare, Sistema Veterinario

Referencia de instalación e integración basada en el código vigente del repositorio

---

Asignatura: Proyecto, Diseño e Implementación de Sistemas Computacionales

Grupo: A 7°4°

Alumna: Banegas Silvia Agustina

Contenido

1. Introducción

2.Convenciones

3. Resumen verificado de métodos

4. Rutas públicas e infraestructura

5. Autenticación, registro y OAuth

6. Área personal del cliente

7. Clínica compartida

8. Administración

9. Administración veterinaria

10. Panel veterinario

11. Invitaciones

12. Códigos de error comunes

13. Qué se verificó y qué consultar antes de integrar

1. Introducción

PetCare es un sistema veterinario que expone una API REST desarrollada con Node.js, Express 5 y TypeScript, encargada de gestionar la lógica de negocio, la autorización, las validaciones y las consultas a la base de datos. El backend se encuentra en la carpeta backend/ y utiliza mysql2/promise para comunicarse con una base de datos MariaDB/MySQL alojada en Layerbase.

La API utiliza /api como prefijo principal de sus rutas, trabaja habitualmente con datos en formato JSON UTF-8 y no implementa una versión /v1. Es consumida por la aplicación web y móvil, desarrolladas con Expo, que se comunican con el backend para acceder a las distintas funcionalidades del sistema.

La URL base para trabajar en entorno local es http://localhost:3000/api. En producción, la API se encuentra disponible a través del dominio configurado para el backend en Vercel, seguido de /api. La URL esperada para el entorno productivo es https://pet-care-sistema-veterinario.vercel.app/api.

Este documento funciona como referencia técnica para conocer, probar e integrar la API de PetCare. Describe las rutas disponibles, los métodos HTTP, los roles y permisos necesarios para acceder a cada recurso, las validaciones correspondientes y los datos que se esperan recibir y devolver. La documentación se elaboró tomando como referencia el código vigente del repositorio, incluyendo backend/src/app.ts, las rutas, los controladores, los servicios y los middlewares.

Para empezar a probar la API

El recorrido habitual para realizar las primeras pruebas es:

1. Levantar el backend local siguiendo la guía de instalación. La API quedará disponible en http://localhost:3000/api.
2. Iniciar sesión mediante POST /api/auth/login para obtener un access token.
3. Enviar el token en todas las solicitudes protegidas mediante el encabezado Authorization: Bearer.
4. Cuando el access token expire, renovar la sesión utilizando POST /api/auth/refresh.

Cada sección de esta documentación corresponde a un grupo de rutas con un mismo prefijo. Se incluyen tablas con los métodos, las rutas, los niveles de acceso y sus funcionalidades, junto con ejemplos de solicitudes y respuestas para facilitar su comprensión e implementación.

2. Convenciones

Éxito y autenticación

La mayoría de controladores responde `{ "success": true, "data": ... }`. Registro suele responder HTTP 201; los listados/consultas, HTTP 200. El encabezado para endpoints protegidos es:

```
Authorization: Bearer <accessToken>
Content-Type: application/json
```
Los endpoints que reciben bytes de comprobante aceptan `application/pdf`, `image/jpeg`, `image/png` o `image/webp`, hasta 5 MiB y usan `X-Payment-Reference` opcional en lugar de JSON. Algunos uploads de foto aceptan JSON con Base64 y hasta 8 MiB.

Método de consulta

No convertir las consultas `POST` a `GET`: solo las rutas explícitas en el inventario aceptan GET. Las rutas `query`, `catalog`, `home`, etc. reciben un objeto JSON, usualmente filtros/paginación; los nombres exactos dependen de la operación delegada a `client.service.ts` o `clinic.service.ts`. No hay un único esquema de filtros compartido para todos los módulos.

Un `POST` de lectura no modifica datos. Los IDs y los discriminadores (`:id`, `:kind`) viajan en la ruta y las rutas `*/query` separan la lectura de colecciones del `POST` de creación. El access token es un JWT de corta duración (900 segundos por defecto); `/auth/refresh` trabaja con la sesión de refresh que administra la capa de autenticación.

Permisos

`AUTH` significa access JWT vigente. `CLIENT personal` significa JWT y acceso personal aplicado; los repositorios derivan owner del usuario autenticado, no de un owner suministrado. `ROLE(A,B)` exige uno de esos roles. `PERM(a,b)` exige **todos** los permisos enumerados; `ANY(a,b)` exige al menos uno. En rutas delegadas, el servicio/repositorio puede aplicar además el grant y el alcance del recurso; consultar la decisión final con la cuenta real. Los cinco roles de aplicación son `CLIENT`, `VETERINARIAN`, `GROOMER`, `ADMIN`, `SUPER_ADMIN`.

Roles del sistema

Rol	Para qué se usa
`CLIENT`	Dueño de mascotas. Usa el área personal y la tienda.
`VETERINARIAN`	Profesional veterinario. Atiende pacientes y carga registros clínicos.
`GROOMER`	Profesional de peluquería. Trabaja con turnos y con la ficha de peluquería.
`ADMIN`	Administrador. Gestiona usuarios, catálogos y configuración según sus permisos.
`SUPER_ADMIN`	Administrador con alcance ampliado. Algunas operaciones, como revisar pagos o reenviar invitaciones, son exclusivas de este rol.

El rol decide qué router se puede usar; los permisos efectivos de cada usuario (por ejemplo `users.manage`), cargados del catálogo MySQL, limitan cada operación dentro de ese router.

3. Autenticación

Las rutas de auth envían `Cache-Control: no-store`. Login/registro usan limitador de solicitudes. Las validaciones estrictas rechazan propiedades extra en registro/login. Registro crea un `CLIENT` sin sesión, luego genera y envía verificación por correo. Login entrega sesión solo si cuenta está activa, verificada y no requiere cambio inicial de contraseña.

Método y ruta	Auth	Request principal	Respuesta/errores relevantes
POST `/api/auth/register`	Público; rate limit	`{firstName,lastName,email,phone?,password}`. No enviar `role` ni confirmación de contraseña.	`201 {success,data:{user,verification}}`; `409 EMAIL_ALREADY_EXISTS`; `400 VALIDATION_ERROR`; `503 EMAIL_UNAVAILABLE`.
POST `/api/auth/login`	Público; rate limit	`{email,password}`	`200 {success,data:session}`; `401 INVALID_CREDENTIALS`; `403 EMAIL_NOT_VERIFIED` o `PASSWORD_CHANGE_REQUIRED`; `503 AUTH_UNAVAILABLE`.
`POST /api/auth/me`	AUTH	Sin campos requeridos	`200 {success:true,data:{user}}`; 401 si sesión no válida.
POST `/api/auth/refresh`	Refresh token en body; no access requerido	`{refreshToken}`	`200 {success,data:<nueva sesión>}`; refresh rotativo/revocable; error auth si vencido o revocado.
POST `/api/auth/logout`	Refresh token en body	`{refreshToken}`	`200 {success,data:{revoked:true}}`; revoca esa sesión.
POST `/api/auth/verify-email`	Público; código/token	`{verificationToken,code}`; token hexadecimal de 64 caracteres, código de 6 dígitos	`200 {success,data:{verified:true,session?}}`; límite de intentos; errores `VERIFICATION_CODE_INVALID`, `VERIFICATION_CODE_EXPIRED`.
POST `/api/auth/resend-verification`	Público; rate limit	`{verificationToken}` hexadecimal de 64 caracteres	`200 {success,data:{verification:{verificationToken,maskedEmai`

Método y ruta	Auth	Request principal	Respuesta/errores relevantes
			`l,retryAfterSeconds,expiresAt,delivery}}}`; espera 45 s y límite por ventana.
POST `/api/auth/password-reset/request`	Público; límite 5	`{email}`	`200 {success,data:{accepted:true}}`; respuesta uniforme para no revelar si existe cuenta.
POST `/api/auth/password-reset/confirm`	Público; límite 10	`{token,password}`; nueva contraseña con reglas de fortaleza	`200 {success,data:{reset:true}}`; token de un uso, vence en 30 min; invalida sesiones previas.
POST `/api/auth/change-initial-password`	Cuenta invitada	`{email,currentPassword,newPassword}`; contraseña nueva con reglas de fortaleza	`200 {success,data:{changed:true}}`; cuenta activa/verificada y marcada para cambio inicial.
GET `/api/auth/oauth/providers?returnUrl=<URL>`	Público; límite	Query `returnUrl` obligatorio, máx. 2048 chars	`{success,data:{providers:{google,facebook,x:boolean}}}`; solo expone disponibilidad.
POST `/api/auth/oauth/:provider/start`	Público; proveedor `google|facebook|x`	`{returnUrl,challenge}`; retorno allowlisted, challenge PKCE URL-safe de 43 caracteres	`{success,data:{configured:true,url}}` para comenzar redirección; `400 VALIDATION_ERROR` o disponibilidad falsa.
GET `/api/auth/oauth/:provider/callback`	Callback del proveedor	Query OAuth `state`, `code` o `error`	Redirección 303 al retorno permitido; no devuelve tokens JWT en URL, sino ticket de un uso.
POST `/api/auth/oauth/exchange`	Público, ticket de un uso	`{ticket,proof}`	`{success,data:{session}}`, `{registrationRequired:true,...}` o `{linked:true}`; ticket

Método y ruta	Auth	Request principal	Respuesta/errores relevantes
			inválido/vencido/usado se rechaza.
POST `/api/auth/oauth/register`	Público	`{ticket,proof,profile:{firstName,lastName,email,phone?,password}}`	`201 {success,data:{verification:...}}`; alta CLIENT requiere email verificable.
POST `/api/auth/oauth/:provider/link`	AUTH	`{returnUrl,challenge}`	`{success,data:{configured:true,url}}` inicia vínculo; se revalida familia de sesión al intercambiar ticket.
POST `/api/auth/invitations/accept`	Token de invitación en body	`{token,password}`; contraseña cumple fuerza requerida	`{success,data:{accepted:true}}`; token aleatorio de un uso, 24 h.
POST `/api/auth/veterinarian-invitations/accept`	Alias de la ruta anterior	Igual que la ruta anterior	Igual; ambas están montadas sobre el mismo router.

Los tokens de invitación y verificación no son access JWT. No copiar tokens reales a logs/documentación. `BACKEND_URL` define el callback HTTPS del proveedor; `OAUTH_RETURN_URLS` define destinos de retorno autorizados (incluido `petcare://oauth` para nativo). `PUBLIC_FRONTEND_URL` define los enlaces web de invitación/recuperación.

Orden habitual de los flujos

- **Alta con correo:** `register`, luego `verify-email` con el código recibido (se puede pedir otro con `resend-verification`) y por último `login`.
- **Recuperar contraseña:** `password-reset/request` envía el correo y `password-reset/confirm` fija la nueva contraseña con el token recibido.
- **Alta por invitación:** la persona recibe un token y usa `invitations/accept` para definir su contraseña; `change-initial-password` cubre las cuentas invitadas con cambio inicial pendiente.
- **Inicio de sesión social:** `oauth/:provider/start` abre la autorización, el proveedor vuelve por el callback `GET` y la app canjea el ticket de un solo uso con `oauth/exchange`.

Las claves concretas de cada DTO están en los controladores y validadores de auth; la API no acepta credenciales OAuth privadas desde la app.

Ejemplos curl de autenticación

```
curl -X POST 'http://localhost:3000/api/auth/login' \
  -H 'Content-Type: application/json' \
  -d '{"email":"ana.ejemplo@example.test","password":"Ejemplo-seguro-42!"}'
```

```
curl -X POST 'http://localhost:3000/api/auth/me' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' \
  -H 'Content-Type: application/json' -d '{}'
```

4. Usuarios, roles y permisos

La administración se encuentra en `/api/admin` y exige sesión y grants. `ADMIN` puede administrar usuarios/roles/permisos según sus permisos asignados; operaciones críticas restringen a `SUPER_ADMIN`. Baja de usuario es lógica, incrementa `session_version`, invalida sesiones y registra auditoría de acción sensible. No hay endpoint de borrado físico de usuarios.

Función	Rutas	Restricción principal
Usuarios y permisos	`/api/admin/users*`	`users.manage`; asignación requiere además roles/permisos según operación. Baja es solo SUPER_ADMIN.
Roles custom	`/api/admin/roles*`	`roles.manage` y `permissions.manage`; crear requiere SUPER_ADMIN; solo se eliminan roles custom sin usuarios.
Catálogo de permisos	`/api/admin/permissions/query`, `/api/admin/catalog`	`permissions.manage` o algún grant de catálogo.
Invitaciones	`/api/admin/users/:id/invitation`; `/api/admin/veterinarians` y rutas de invitación	Requiere grant de usuarios/profesionales; reenvío de invitación de usuario requiere SUPER_ADMIN.
Veterinarios	`/api/admin/veterinarians*`	`ADMIN`/`SUPER_ADMIN` más `veterinarians.manage`.
Auditoría	Operaciones sensibles llaman servicio/repository de auditoría	La auditoría se registra en el backend; el repositorio no define un endpoint general de consulta de auditoría.

No se deben inferir grants únicamente por el rol. `requirePermission` exige todos los códigos dados; `requireAnyPermission`, alguno. Los datos de permisos se recargan desde base al autenticar. El módulo puede imponer ownership o alcance dentro del repositorio.

5. Módulos funcionales

Las operaciones de cliente están bajo `/api/client`; cada request requiere JWT y guard de acceso personal. El backend toma el ID del cliente de la sesión. Las operaciones de clínica están bajo `/api/clinic` y requieren rol/grants enumerados en el inventario. Los catálogos administrativos genéricos usan `:kind`; valores admitidos dependen de `clinic.service.ts`/`admin.repository.ts` y no se deben adivinar desde el path.

Los módulos verificados incluyen mascotas/catálogo, turnos y slots, fichas clínicas por tipo, clientes y profesionales, especialidades, disponibilidad, tienda/categorías/promociones/banners, productos e

inventario, fulfillment/direcciones, carrito, pedidos, transferencia/comprobante y revisión administrativa, perfiles, información general, notificaciones y fotos protegidas. No hay integración de pago con procesador externo confirmada; `CARD_DEMO` es simulación no cobrable.

Pedidos: demostración y transferencia

Hay dos maneras de crear un pedido. El checkout de demostración (POST /api/client/orders) solo funciona con STORE_DEMO_MODE=true y no procesa dinero. El pedido por transferencia (POST /api/client/orders/bank-transfer) crea una solicitud de transferencia manual; no cobra automáticamente. El cliente sube el comprobante con POST /api/client/payment-intents/:id/proof y, si hace falta, renueva la intención con POST /api/client/orders/:id/payment-intents. El pago requiere revisión administrativa.

Entradas conocidas del área cliente

Ruta	Entrada
`POST /api/client/pets/query`	`{includeInactive?}`, booleano o string.
`POST /api/client/appointments/slots`	`{serviceId, veterinarianId?}`, IDs positivos.
`POST /api/client/appointments`	`{petId, slotId}`; valida la disponibilidad del slot; responde 201.
`POST /api/client/clinical/:kind`	`kind`: `medical-history`, `vaccines`, `prescriptions` o `recommendations`; `{petId?}`; solo lectura.
`POST /api/client/store/delivery-quote`	`{province, city, postalCode, subtotalCents}`.
`POST /api/client/store/products`	`{search?, categoryId?, productType?}`; `productType`: `MEDICINE` o `ACCESSORY`.
`PUT /api/client/cart/:id`	`{quantity}`.
`POST /api/client/notifications`	`{cursor?, limit?}`.
`PATCH /api/client/profile`	`{firstName, lastName, phone}`.
`PUT /api/client/profile/password`	`{currentPassword, password}`; límite de tasa.

Notificaciones

`POST /api/notifications` consulta el centro; `PATCH /read-all`, `PATCH /:id/read` y `DELETE /:id` modifican estado personal. El alias `/api/client/notifications` conserva las mismas cuatro operaciones y el guard personal del cliente. Notificaciones se guardan en DB; el job puede crear recordatorios internos y despachar correo. No hay push móvil/web implementado.

Notificaciones y health públicos

`GET /api/health` prueba el proceso; `GET /api/health/database` verifica motor SQL; `GET /api/settings/general-information` publica datos generales con cache de 5 minutos. `GET /api/cron/notifications` requiere Bearer `CRON_SECRET`. El router del cron existe, pero `backend/vercel.json` está vacío y no configura programación automática en este checkout.

6. Contratos de datos

Reglas confirmadas por validadores

Dato	Regla observada
IDs de cliente	JSON string numérico positivo de hasta 20 dígitos, dentro de rango unsigned BIGINT.
Registro	`firstName`/`lastName`: texto normalizado de hasta 100 caracteres, letras/apóstrofes/espacios/guiones; `email` normalizado minúscula, TLD requerido y hasta 254; `phone` opcional, 7–15 dígitos; password min. 8 caracteres, mayúscula, minúscula, número, especial y máx. 72 bytes UTF-8.
Mascota	`name`, `sex` enum `FEMALE|MALE|UNKNOWN`, `speciesId`, opcional `breedId`, `breedName`, `birthDate` (`YYYY-MM-DD`, no futuro), `microchipNumber`, URL foto HTTPS, `photoBase64`, `weightKg` >0 hasta 9999.99 con dos decimales. Se rechazan campos extra.
Carrito	`quantity` entero entre 0 y 99; 0 puede vaciar/eliminar unidad según endpoint.
Idempotencia	`requestKey` es UUID v4.
Checkout	`paymentMethod`: `BANK_TRANSFER|CARD_DEMO`; `fulfillmentMethod`: `PICKUP|DELIVERY`. Delivery requiere dirección con destinatario, teléfono, calle, número, ciudad, provincia y código postal; unidad/referencia opcionales.
Transferencia	Añade `buyerName`, `buyerEmail`, `buyerDni`; el DNI se normaliza/valida y se descarta antes de persistir la intención.
Instantes de clínica	Deben incluir zona `Z` u offset (`YYYY-MM-DDTHH:mm:ss[.sss]±hh:mm`); se convierten a UTC.
Paginación	El validador de clínica admite `page` como entero decimal positivo de hasta 5 dígitos. Cada módulo puede añadir filtros propios.
Foto de perfil	Body permite solo `photoBase64` o `{removePhoto:true}`; no ambos. Upload recibe JSON y límite middleware 8 MiB.

Muchos contratos se validan en los servicios y repositorios por operación, no en un schema global único. Para filtros y catálogos usar el contrato de la pantalla/API correspondiente y revisar el validator del módulo; no asumir que todas las consultas aceptan los mismos campos, ordenamiento o paginación. Fechas persistidas se interpretan en UTC. IDs SQL grandes se exponen como string para evitar pérdida de precisión JS.

7. Respuestas y errores

Forma estándar de error:

```
{
  "success": false,
```

```
  "error": {
    "code": "VALIDATION_ERROR",
    "message": "Revisá los datos ingresados.",
    "errors": [{ "field": "email", "message": "Ingresá un email válido." }]
  }
}
```

`errors` aparece solo si hay errores de campo; `verification` puede aparecer en flujos de verificación. No todos los errores incluyen ambos. Formato común de éxito: `{ "success": true, "data": ... }`; algunas rutas públicas específicas lo fijan de forma distinta solo en el contenido `data`.

HTTP	Código interno observado / uso
400	`VALIDATION_ERROR`, `INVALID_JSON`, `INVALID_INVITATION`, `INVALID_*`; JSON mal formado se transforma en `INVALID_JSON`.
401	`AUTH_REQUIRED`, `INVALID_CREDENTIALS`; token ausente/inválido o login fallido.
403	`FORBIDDEN`, `EMAIL_NOT_VERIFIED`, `PASSWORD_CHANGE_REQUIRED`.
404	`NOT_FOUND`; para ruta ausente de tienda `ROUTE_NOT_FOUND`.
409	`EMAIL_ALREADY_EXISTS`, `DB_CONFLICT`, `DB_RELATION_CONFLICT`, `PAYMENT_METHOD_UNAVAILABLE`, `INVITATION_NOT_AVAILABLE`, `RECORD_IN_USE`.
410	`VERIFICATION_CODE_EXPIRED` cuando venció el código de verificación.
413	`PAYLOAD_TOO_LARGE` para cuerpo por encima del límite.
429	rate limit; también `INVITATION_COOLDOWN` para reenvío demasiado frecuente.
500	`INTERNAL_ERROR`, `DB_ERROR`, `SCHEMA_UPDATE_REQUIRED`.
503	`DB_UNAVAILABLE`, `AUTH_UNAVAILABLE`, `EMAIL_UNAVAILABLE`, `OAUTH_UNAVAILABLE`, `NOTIFICATION_JOB_UNAVAILABLE`, `PHOTO_STORAGE_UNAVAILABLE`.

El validador no produce un `422` general: las entradas inválidas observadas usan 400. El código define limitadores, pero no se fija un código interno `429` universal. Los errores SQL se normalizan y no exponen SQL, credenciales ni stack trace al cliente. Duplicados se normalizan a conflicto. Campos/constraints desconocidos requieren confirmar el esquema antes de actuar.

Límites de carga: 8 MB para fotos y 5 MB para comprobantes; superarlos responde `413 PAYLOAD_TOO_LARGE`. `401 AUTH_REQUIRED` también aparece cuando la credencial del cron no coincide, y `404 ROUTE_NOT_FOUND` indica que el método o la ruta no existen: verificá la URL base y usá el método vigente.

8. Seguridad

- JWT access usa `JWT_ACCESS_SECRET` (mínimo 32 bytes), issuer/audience y TTL configurable hasta 3600 s. Sesiones refresh se almacenan y revocan en backend; cerrar sesión revoca el refresh enviado.
- Las contraseñas se hashean con bcrypt. El login hace comparación dummy si el email no existe para reducir diferencias temporales.
- El middleware vuelve a cargar cuenta, roles y permisos; verifica cuenta activa, versión de sesión, verificación y cambios iniciales pendientes.

- Tokens de invitación/reset son aleatorios, almacenados como hash, de un uso y con vencimiento. Una invitación dura 24 h; reenviar rota el token.
- Ownership se deriva del JWT. Rutas clínicas y comerciales revalidan permisos/alcance, algunas dentro de transacción/repository.
- Endpoints de autenticación tienen rate limits; algunos uploads también.
- CORS permite el sitio oficial/previews configurados y orígenes exactos; producción solo HTTPS. Helmet está habilitado. Las respuestas de auth/privadas usan `no-store`.
- El log del middleware oculta SQL y valores de parámetros; el cliente recibe mensajes controlados.
- Las bajas administrativas son lógicas y auditadas; operaciones sensibles pueden depender de la existencia del esquema de auditoría en la DB.

9. Integraciones externas

Integración	Estado en el código
Layerbase / MySQL / MariaDB	Conexión SQL real por pool; se requiere host de entorno y TLS en producción. Compatibilidad comprobada por el chequeo de versión.
Vercel	Configuración de salida web y backend; dominios citados son esperados, no evidencia de estado remoto. `backend/vercel.json` está vacío en checkout.
SMTP	Nodemailer para verificación, recuperación, invitaciones y correo de notificación; configuración opcional por entorno, necesaria para estos flujos.
Google, Facebook, X	OAuth servidor. Credenciales y callback exacto deben configurarse en proveedor. X mantiene fallback de nombre de variables `TWITTER_*`.
Cloudinary / S3	Servicios alternativos de fotos; habilitar credenciales privadas. URL de foto del animal puede servirse mediante `/media/pets/:id`, que verifica ownership.
Pago	No se ve un procesador real. `CARD_DEMO` es modo de demostración opcional; `BANK_TRANSFER` usa intención y carga/revisión de comprobante, no acredita el pago por sí misma.
Cron de notificaciones	Servicio y endpoint implementados; configuración automática no comprobada: `backend/vercel.json` es `{}`. Invocación debe protegerse con Bearer `CRON_SECRET`.
Expo Push / Web Push	No implementado. Notificaciones internas persisten y se consultan por API.

10. Ejemplos prácticos

Todos los datos de ejemplo son ficticios. Reemplazar `<ACCESS_TOKEN>` y los IDs por datos de la propia respuesta.

Crear una mascota

```
curl -X POST 'http://localhost:3000/api/client/pets' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' \
  -d
'{"name":"Luna","sex":"FEMALE","speciesId":"1","breedId":null,"breedName":"Mestiza
","birthDate":"2022-04-12","microchipNumber":null,"photoUrl":null,"weightKg":8.4}'
```

Consultar mascotas

```
curl -X POST 'http://localhost:3000/api/client/pets/query' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' -d
'{}'
```

Consultar turnos y crear un turno

El catálogo/slots debe consultarse primero; use IDs y campos de slot/servicio devueltos por ese catálogo. La operación valida disponibilidad en el backend.

```
curl -X POST 'http://localhost:3000/api/client/appointments/slots' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' \
  -d '{"serviceId":"1","veterinarianId":"2"}'
```

```
curl -X POST 'http://localhost:3000/api/client/appointments' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' \
  -d '{"petId":"1","slotId":"12"}'
```

Los campos concretos se validan por `client.service.ts`; el ejemplo ilustra el flujo y no sustituye valores retornados por el catálogo del entorno.

Crear/aceptar invitación

La creación administrativa depende del recurso y grants; el ejemplo crea una cuenta desde la administración. El enlace se envía por correo; no se devuelve ni se debe registrar el token.

```
curl -X POST 'http://localhost:3000/api/admin/users' \
  -H 'Authorization: Bearer <ADMIN_ACCESS_TOKEN>' -H 'Content-Type:
application/json' \
  -d
'{"firstName":"Mateo","lastName":"Ejemplo","email":"mateo@example.test","roleId":"
2"}'
```

```
curl -X POST 'http://localhost:3000/api/auth/invitations/accept' \
  -H 'Content-Type: application/json' \
  -d '{"token":"<TOKEN_RECIBIDO_POR_CORREO>","password":"Nueva-Clave-42!"}'
```

Consultar productos

```
curl -X POST 'http://localhost:3000/api/client/store/products' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' -d
'{"search":"","productType":"ACCESSORY"}'
```

Crear pedido

El checkout requiere UUID v4 para idempotencia, un carrito y método/entrega disponibles en la configuración de la tienda. El modo tarjeta demo no procesa dinero y `/api/client/orders` responde `409 STORE_DEMO_DISABLED` mientras `STORE_DEMO_MODE` permanezca en `false` (default). Para una prueba real, usar un entorno de desarrollo autorizado con carrito y modo demo habilitado; no probar pedidos contra producción.

```
curl -X POST 'http://localhost:3000/api/client/orders' \
  -H 'Authorization: Bearer <ACCESS_TOKEN>' -H 'Content-Type: application/json' \
  -d
'{"requestKey":"2a0e83dd-d290-4bf3-9af7-a1d5939eb510","paymentMethod":"CARD_DEMO",
"paymentReference":"","fulfillmentMethod":"PICKUP"}'
```

El contrato exacto y los campos obligatorios los define el validador de checkout en `backend/src/validators/`; consultalo al integrar un cliente nuevo. La demostración no procesa dinero.

Login

```
curl -X POST 'http://localhost:3000/api/auth/login' \
  -H 'Content-Type: application/json' \
  -d '{"email":"cliente@example.test","password":"Ejemplo-seguro-42!"}'
```

11. Inventario completo de endpoints

El inventario enumera todas las declaraciones de ruta encontradas en `backend/src/routes`, `backend/src/routes/index.ts` y rutas adicionales de `backend/src/app.ts`. Los comodines `:id`, `:kind`, `:provider` son segmentos dinámicos. Cuando hay alias, cada URL se lista por separado. Los permisos de los routers genéricos pueden volver a comprobarse en los servicios/repositorios; una celda `dinámico` no autoriza por sí sola la operación.

Infraestructura y rutas públicas

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
GET	`/api/health`	Salud	No	Ninguno	`backend/src/routes/index.ts`
GET	`/api/health/database`	Salud SQL	No	Ninguno	`backend/src/routes/index.ts`
GET	`/api/settings/general-information`	Configuración pública	No	Ninguno	`backend/src/routes/settings.routes.ts`
GET	`/api/cron/notifications`	Jobs	Bearer `CRON_SECRET`	Secreto cron	`backend/src/routes/notification-jobs.routes.ts`
GET	`/media/pets/:id`	Foto de mascota autorizada	JWT	Acceso personal / ownership	`backend/src/app.ts`

`/api/health` solo confirma que Express responde y no verifica MariaDB; para comprobar también la conexión SQL usá `/api/health/database`. Ejemplo:

```
GET /api/health
```

```
{"success":true,"data":{"status":"ok","message":"PetCare API funcionando"}}
```

`GET /api/settings/general-information` usa caché pública de 300 s y no expone datos de administración. `GET /api/cron/notifications` responde `503 NOTIFICATION_JOB_UNAVAILABLE` si falta un `CRON_SECRET` de al menos 32 bytes y `401 AUTH_REQUIRED` si el Bearer no coincide. `GET /media/pets/:id` exige JWT, acceso personal y límite de tasa, devuelve una URL autorizada `{url}` con `Cache-Control: no-store` y no acepta una clave de almacenamiento arbitraria. El callback OAuth redirige al destino permitido de PetCare: los errores se expresan en el retorno y no es una lectura de API para clientes.

Auth y OAuth

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/auth/register`	Auth	No	Ninguno	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/login`	Auth	No	Ninguno	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/password-reset/request`	Recuperación	No	Ninguno	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/password-reset/confirm`	Recuperación	Token reset	Token válido	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/change-initial-password`	Cuenta invitada	Token/cambio inicial	Flujo de invitación	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/verify-email`	Verificación	No	Código/token válido	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/resend-verification`	Verificación	No	Cuenta/flujo válido	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/me`	Sesión	JWT	Usuario autenticado	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/refresh`	Sesiones	Refresh token	Refresh vigente	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/logout`	Sesiones	Refresh token	Sesión revocable	`backend/src/routes/auth.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/auth/oauth/exchange`	OAuth	Ticket de un uso	Ticket vigente	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/oauth/register`	OAuth	Ticket/estado OAuth	Política OAuth	`backend/src/routes/auth.routes.ts`
GET	`/api/auth/oauth/providers`	OAuth	No	Query `returnUrl` allowlisted	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/oauth/:provider/start`	OAuth	No	Provider y retorno configurados	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/oauth/:provider/link`	OAuth vínculo	JWT	Cuenta autenticada	`backend/src/routes/auth.routes.ts`
GET	`/api/auth/oauth/:provider/callback`	OAuth callback	Proveedor OAuth	State válido	`backend/src/routes/auth.routes.ts`
POST	`/api/auth/invitations/accept`	Invitaciones	Token invitación	Token vigente	`backend/src/routes/veterinarian.routes.ts`
POST	`/api/auth/veterinarian-invitations/accept`	Invitaciones (alias)	Token invitación	Token vigente	`backend/src/routes/veterinarian.routes.ts`

Cliente: cuenta, mascotas y turnos

Todas las rutas en esta subsección requieren JWT y guard de acceso personal. El backend deriva la persona propietaria del token.

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/client/home`	Inicio	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/pets/catalog`	Mascotas/catálogos	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/pets/query`	Mascotas	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/pets`	Mascotas	JWT	Acceso personal	`backend/src/routes/client.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/client/pets/:id`	Mascotas	JWT	Ownership	`backend/src/routes/client.routes.ts`
PUT	`/api/client/pets/:id`	Mascotas	JWT	Ownership	`backend/src/routes/client.routes.ts`
PATCH	`/api/client/pets/:id/deactivate`	Mascotas	JWT	Ownership	`backend/src/routes/client.routes.ts`
PATCH	`/api/client/pets/:id/reactivate`	Mascotas	JWT	Ownership	`backend/src/routes/client.routes.ts`
POST	`/api/client/appointments/catalog`	Turnos	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/appointments/slots`	Agenda	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/appointments/query`	Turnos	JWT	Ownership	`backend/src/routes/client.routes.ts`
POST	`/api/client/appointments/:id`	Turnos	JWT	Ownership	`backend/src/routes/client.routes.ts`
PATCH	`/api/client/appointments/:id/cancel`	Turnos	JWT	Ownership / estado cancelable	`backend/src/routes/client.routes.ts`
POST	`/api/client/appointments`	Turnos	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/clinical/:kind`	Historial clínico	JWT	Acceso personal y regla del servicio	`backend/src/routes/client.routes.ts`

Cliente: tienda, pedidos y perfil

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/client/store/categories`	Tienda/categorías	JWT	Acceso personal	`backend/src/routes/client.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/client/store/fulfillment`	Tienda/entrega	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/promotions`	Tienda/promociones	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/banners`	Tienda/banners	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/delivery-quote`	Envíos	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/addresses/query`	Direcciones	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/addresses`	Direcciones	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
PATCH	`/api/client/store/addresses/:id`	Direcciones	JWT	Ownership	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/products`	Tienda/productos	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/store/products/:id`	Tienda/producto	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
POST	`/api/client/cart`	Carrito	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
PUT	`/api/client/cart/:id`	Carrito	JWT	Ownership	`backend/src/routes/client.routes.ts`
POST	`/api/client/orders/query`	Pedidos	JWT	Ownership	`backend/src/routes/client.routes.ts`
POST	`/api/client/orders`	Checkout/pedido	JWT	Acceso personal, carrito válido	`backend/src/routes/client.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/client/orders/bank-transfer`	Transferencias	JWT	Acceso personal; rate limit	`backend/src/routes/client.routes.ts`
POST	`/api/client/orders/:id/payment-intents`	Intención de pago	JWT	Ownership; rate limit	`backend/src/routes/client.routes.ts`
POST	`/api/client/payment-intents/:id/proof`	Comprobante	JWT	Ownership; rate limit	`backend/src/routes/client.routes.ts`
POST	`/api/client/orders/:id`	Pedido	JWT	Ownership	`backend/src/routes/client.routes.ts`
PATCH	`/api/client/profile`	Perfil	JWT	Acceso personal	`backend/src/routes/client.routes.ts`
PUT	`/api/client/profile/password`	Perfil/seguridad	JWT	Acceso personal; rate limit	`backend/src/routes/client.routes.ts`

Notificaciones

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/notifications`	Notificaciones	JWT	Rol habilitado para centro	`backend/src/routes/notification.routes.ts`
PATCH	`/api/notifications/read-all`	Notificaciones	JWT	Rol habilitado para centro	`backend/src/routes/notification.routes.ts`
PATCH	`/api/notifications/:id/read`	Notificaciones	JWT	Ownership	`backend/src/routes/notification.routes.ts`
DELETE	`/api/notifications/:id`	Notificaciones	JWT	Ownership	`backend/src/routes/notification.routes.ts`
POST	`/api/client/notifications`	Notificaciones (alias cliente)	JWT	Acceso personal	`backend/src/routes/notification.routes.ts`
PATCH	`/api/client/notifications/read-all`	Notificaciones (alias cliente)	JWT	Acceso personal	`backend/src/routes/notification.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
PATCH	`/api/client/notifications/:id/read`	Notificaciones (alias cliente)	JWT	Acceso personal / ownership	`backend/src/routes/notification.routes.ts`
DELETE	`/api/client/notifications/:id`	Notificaciones (alias cliente)	JWT	Acceso personal / ownership	`backend/src/routes/notification.routes.ts`

Clínica, pacientes y turnos profesionales

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/clinic/home`	Inicio clínica	JWT, rol profesional/admin	`VETERINARIAN/GROOMER/ADMIN/SUPER_ADMIN`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/catalog`	Catálogos	JWT	Rol `CLIENT`/profesional/admin	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/products/query`	Productos profesionales	JWT	rol veterinario/admin y `ANY(products.view,prescriptions.create,recommendations.create)`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/slots`	Agenda	JWT	rol profesional/admin y `ANY(appointments.manage,schedule.manage)`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/appointments/:id`	Turno	JWT	profesional/admin y `ANY(appointments.view_own,appointments.view_all)`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/appointments`	Turnos	JWT	profesional/admin y `ANY(appointments.view_own,appointments.view_all)`	`backend/src/routes/clinic.routes.ts`
PATCH	`/api/clinic/appointments/:id`	Turno	JWT	profesional/admin y `ANY(appointments`	`backend/src/routes/clinic.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
				.update_own,appointments.manage)	
POST	/api/clinic/grooming/appointments/:id/record/query	Registro peluquería	JWT, GROOMER	appointments.view_own	backend/src/routes/clinic.routes.ts
PUT	/api/clinic/grooming/appointments/:id/record	Registro peluquería	JWT, GROOMER	appointments.update_own	backend/src/routes/clinic.routes.ts
POST	/api/clinic/patients/query	Pacientes	JWT, profesional/admin	ANY(pets.view_information,appointments.view_own)	backend/src/routes/clinic.routes.ts
POST	/api/clinic/patients/:id	Paciente	JWT, profesional/admin	ANY(pets.view_information,appointments.view_own)	backend/src/routes/clinic.routes.ts
PATCH	/api/clinic/patients/:id	Paciente	JWT, veterinario/admin	pets.update	backend/src/routes/clinic.routes.ts
PATCH	/api/clinic/patients/:id/deactivate	Paciente	JWT, profesional/admin	pets.update	backend/src/routes/clinic.routes.ts
PATCH	/api/clinic/patients/:id/reactivate	Paciente	JWT, profesional/admin	pets.update	backend/src/routes/clinic.routes.ts
POST	/api/clinic/patients/:id/records/:kind/query	Historia clínica	JWT, veterinario/admin	Rol solamente en ruta; autorización de recurso/servicio	backend/src/routes/clinic.routes.ts
POST	/api/clinic/patients/:id/records/:kind/page	Historia clínica/página	JWT, veterinario/admin	Rol solamente en ruta; autorización de recurso/servicio	backend/src/routes/clinic.routes.ts
POST	/api/clinic/patients/:id/records/:kind	Crear registro clínico	JWT, veterinario; rate limit	Rol veterinario y autorización del servicio	backend/src/routes/clinic.routes.ts
POST	/api/clinic/clients/query	Clientes	JWT, veterinario/admin	Rol solamente en ruta; autorización del servicio	backend/src/routes/clinic.routes.ts

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/clinic/clients/:id`	Cliente	JWT, veterinario/admin	Rol solamente en ruta; autorización del servicio	`backend/src/routes/clinic.routes.ts`

Profesionales, agenda y catálogos de clínica

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/clinic/professionals`	Profesionales/servicios	JWT, admin	`professionals.manage`	`backend/src/routes/clinic.routes.ts`
PUT	`/api/clinic/professionals/:id/type`	Tipo profesional	JWT, admin	`professionals.manage`	`backend/src/routes/clinic.routes.ts`
PUT	`/api/clinic/professionals/:id/services`	Servicios del profesional	JWT, admin	`professionals.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/availability/query`	Disponibilidad	JWT, admin	`schedule.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/availability`	Disponibilidad	JWT, admin	`schedule.manage`	`backend/src/routes/clinic.routes.ts`
PATCH	`/api/clinic/availability/:kind/:id`	Desactivar disponibilidad	JWT, admin	`schedule.manage`	`backend/src/routes/clinic.routes.ts`
DELETE	`/api/clinic/availability/:kind/:id`	Archivar disponibilidad	JWT, admin	`schedule.manage`	`backend/src/routes/clinic.routes.ts`

Administración de tienda, órdenes y catálogos

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/clinic/store-fulfillment`	Configuración de entrega	JWT, admin	`store.settings.view`	`backend/src/routes/clinic.routes.ts`
PUT	`/api/clinic/store-fulfillment`	Configuración de entrega	JWT, admin	`store.settings.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-banners/query`	Banners	JWT, admin	`store.banners.manage`	`backend/src/routes/clinic.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
PUT	`/api/clinic/store-banners/order`	Orden de banners	JWT, admin	`store.banners.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-banners`	Crear banner	JWT, admin	`store.banners.manage`	`backend/src/routes/clinic.routes.ts`
PATCH	`/api/clinic/store-banners/:id`	Editar banner	JWT, admin	`store.banners.manage`	`backend/src/routes/clinic.routes.ts`
DELETE	`/api/clinic/store-banners/:id`	Eliminar banner	JWT, admin	`store.banners.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-orders`	Pedidos administrativos	JWT, admin	`orders.view_all`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-orders/:id`	Pedido administrativo	JWT, admin	`orders.view_all`	`backend/src/routes/clinic.routes.ts`
PATCH	`/api/clinic/store-orders/:id`	Estado/pedido	JWT, admin	`ANY(orders.manage,orders.payment.manage)`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-orders/:id/proof`	Revisar comprobante	JWT, SUPER_ADMIN	`orders.payment.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/store-orders/:id/payment-review`	Revisión de pago	JWT, SUPER_ADMIN	`orders.payment.manage`	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/admin/:kind/query`	Catálogo admin genérico	JWT	Autorización dinámica en servicio/repository	`backend/src/routes/clinic.routes.ts`
POST	`/api/clinic/admin/:kind`	Crear catálogo admin	JWT; rate limit	Autorización dinámica en servicio/repository	`backend/src/routes/clinic.routes.ts`
PATCH	`/api/clinic/admin/:kind/:id`	Editar catálogo admin	JWT	Autorización dinámica en servicio/repository	`backend/src/routes/clinic.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
DELETE	`/api/clinic/admin/:kind/:id`	Borrar catálogo admin	JWT	Autorización dinámica en servicio/repository	`backend/src/routes/clinic.routes.ts`

Cómo leer la columna de permisos. Los roles operativos son `VETERINARIAN`, `GROOMER`, `ADMIN` y `SUPER_ADMIN`. Cuando se indica un rol más un permiso, hacen falta las dos cosas; cuando aparecen varios permisos con `ANY`, alcanza con uno. Las rutas `POST` de catálogo y de consulta no escriben datos. Los nombres de campos concretos están en `clinic.controller.ts` y `clinic.repository.ts`; no amplíes los cuerpos suponiendo que el repositorio acepte campos arbitrarios.

Administración general, usuarios, roles y productos

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/admin/home`	Inicio administración	JWT, admin	`ADMIN` o `SUPER_ADMIN`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/general-information`	Información general	JWT, admin	`settings.general.manage`	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/general-information`	Actualizar info general	JWT, admin	`settings.general.manage`	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/account`	Cuenta admin propia	JWT, admin	rol admin	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/account/password`	Clave admin propia	JWT, admin	rol admin; rate limit	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/catalog`	Catálogos de seguridad	JWT, admin	`ANY(users.manage,roles.manage,permissions.manage)`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/users/query`	Usuarios	JWT, admin	`users.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/users/:id`	Usuario	JWT, admin	`users.manage`	`backend/src/routes/admin.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
DELETE	`/api/admin/users/:id`	Baja lógica de usuario	JWT, SUPER_ADMIN	`users.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/users`	Crear usuario/invitación	JWT, admin	`users.manage` y `roles.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/users/:id/invitation`	Reenviar invitación	JWT, SUPER_ADMIN	Rol SUPER_ADMIN	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/users/:id`	Actualizar usuario	JWT, admin	`users.manage`	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/users/:id/photo`	Foto usuario	JWT, admin	`users.manage`	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/users/:id/permissions`	Permisos usuario	JWT, admin	`users.manage` y `permissions.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/roles/query`	Roles	JWT, admin	`roles.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/roles`	Crear rol	JWT, SUPER_ADMIN	`roles.manage` y `permissions.manage`	`backend/src/routes/admin.routes.ts`
PUT	`/api/admin/roles/:id`	Editar rol	JWT, admin	`roles.manage` y `permissions.manage`	`backend/src/routes/admin.routes.ts`
DELETE	`/api/admin/roles/:id`	Borrar rol custom	JWT, admin	`roles.manage` y `permissions.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/permissions/query`	Permisos	JWT, admin	`permissions.manage`	`backend/src/routes/admin.routes.ts`
POST	`/api/admin/products/query`	Productos admin	JWT	`ANY(products.update,products.update_stock,produc`	`backend/src/routes/admin.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
				ts.update_price)	
POST	/api/admin/products/catalog	Categorías producto	JWT	ANY(products.update,products.update_stock,products.update_price)	backend/src/routes/admin.routes.ts
POST	/api/admin/products/:id/stock-movements	Movimientos stock	JWT	products.update_stock	backend/src/routes/admin.routes.ts
PATCH	/api/admin/products/:id	Producto/foto	JWT	ANY(products.update,products.update_stock,products.update_price); imagen requiere products.update	backend/src/routes/admin.routes.ts

`requirePermission` exige todos los permisos indicados y `requireAnyPermission`, al menos uno.
Ejemplo de consulta de usuarios:

```
POST /api/admin/users/query
Authorization: Bearer <accessToken>
Content-Type: application/json
```

```
{"search":"ana","page":1,"status":"active"}
```

Las fotos de usuario y de veterinario aceptan `{photoBase64}` o `{removePhoto:true}`.

Veterinarios y especialidades

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	/api/admin/veterinarians/catalog	Catálogo veterinarios	JWT, admin	veterinarians.manage	backend/src/routes/veterinarian.routes.ts
POST	/api/admin/veterinarians/query	Listado veterinarios	JWT, admin	veterinarians.manage	backend/src/routes/veterinarian.routes.ts
POST	/api/admin/veterinarians/:id	Detalle veterinario	JWT, admin	veterinarians.manage	backend/src/routes/veterinarian.routes.ts

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/admin/veterinarians`	Crear veterinario/invitar	JWT, admin	`veterinarians.manage`	`backend/src/routes/veterinarian.routes.ts`
PUT	`/api/admin/veterinarians/:id`	Editar veterinario	JWT, admin	`veterinarians.manage`	`backend/src/routes/veterinarian.routes.ts`
PUT	`/api/admin/veterinarians/:id/photo`	Foto veterinario	JWT, admin	`veterinarians.manage`	`backend/src/routes/veterinarian.routes.ts`
PATCH	`/api/admin/veterinarians/:id/status`	Estado veterinario	JWT, admin	`veterinarians.manage`	`backend/src/routes/veterinarian.routes.ts`
POST	`/api/admin/specialties/query`	Especialidades	JWT, admin	`specialties.manage`	`backend/src/routes/veterinarian.routes.ts`
POST	`/api/admin/specialties`	Crear especialidad	JWT, admin	`specialties.manage`	`backend/src/routes/veterinarian.routes.ts`
PUT	`/api/admin/specialties/:id`	Editar/activar especialidad	JWT, admin	`specialties.manage`	`backend/src/routes/veterinarian.routes.ts`
DELETE	`/api/admin/specialties/:id`	Eliminar especialidad	JWT, admin	`specialties.manage`; conflicto 409 si hay relaciones	`backend/src/routes/veterinarian.routes.ts`

Los listados de veterinarios admiten filtros de búsqueda, estado, orden, página y especialidad; los IDs son numéricos positivos. `PATCH /api/admin/veterinarians/:id/status` requiere `{isActive: boolean}`. En especialidades, `POST` exige `{name}` (texto de hasta 100 caracteres, responde 201) y `PUT` acepta `{name?, isActive?}` con al menos un campo. `DELETE` solo elimina si la especialidad no está asociada: responde `409 RECORD_IN_USE` si tiene servicios o veterinarios relacionados.

Panel profesional y fotos

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
POST	`/api/vet/profile`	Perfil profesional	JWT, VETERINARIAN	Rol VETERINARIAN	`backend/src/routes/veterinarian.routes.ts`
PATCH	`/api/vet/profile`	Editar perfil profesional	JWT, VETERINARIAN	Rol VETERINARIAN	`backend/src/routes/veterinarian.routes.ts`

Método	Endpoint	Módulo	Autenticación	Permiso requerido	Archivo fuente
PUT	`/api/vet/profile/password`	Clave profesional	JWT, VETERINARIAN	Rol VETERINARIAN	`backend/src/routes/veterinarian.routes.ts`
POST	`/api/vet/home`	Inicio panel	JWT, VETERINARIAN	Rol VETERINARIAN; módulos según grants	`backend/src/routes/veterinarian.routes.ts`
POST	`/api/vet/modules/:module`	Módulo profesional	JWT, VETERINARIAN	Permiso de módulo consultado	`backend/src/routes/veterinarian.routes.ts`
PUT	`/api/vet/profile/photo/`	Foto de profesional	JWT, VETERINARIAN o GROOMER	Rol admitido y rate limit	`backend/src/routes/veterinarian.routes.ts`

`POST /api/vet/modules/:module` verifica el acceso al módulo indicado y responde 403 si no está habilitado; los módulos se calculan según los permisos efectivos y el catálogo de permisos.

Conteo: 148 rutas y métodos declarados/montados en este inventario, incluidos los dos alias de invitación y cuatro alias de notificaciones. Los paths dinámicos cuentan como una ruta declarada cada uno; no se expanden todos los posibles valores de `:id`, `:kind` o `:provider`. Este conteo no cuenta respuestas 404 genéricas ni middleware `app.use` como rutas de negocio.

12. Qué se verificó y qué consultar antes de integrar

Se verificaron las rutas registradas, el método, el prefijo, el middleware visible, la operación o handler, los contratos explícitos de `client.service.ts`, los validadores dedicados de autenticación, mascotas, veterinaria y checkout, y los errores normalizados.

Algunos cuerpos de clínica y administración son contratos de operación o de repositorio y se describieron por campo solo cuando el código lo hace explícito. Para esos formularios, inspeccioná el tipo y la validación en el handler o repository de la misma operación antes de enviar campos adicionales. Los ejemplos de respuesta compactos representan el envelope, no una promesa de que las listas estarán vacías.

Fuentes ejecutables principales: `backend/src/routes/`, `backend/src/app.ts`, `backend/src/controllers/`, `backend/src/services/client.service.ts`, `backend/src/repositories/clinic.repository.ts`, `backend/src/validators/` y `backend/src/middlewares/error.middleware.ts`.

13. Estado de esquema y comprobaciones de integración

El código registra acciones administrativas en admin_sensitive_action_audit y cambios de estado de mascotas en pet_lifecycle_history. El repositorio no incluye el SQL canónico de esas tablas ni confirma la estructura instalada en Layerbase. La existencia y compatibilidad del esquema remoto requieren una consulta de solo lectura a information_schema.

El backend inserta en admin_sensitive_action_audit las columnas actor_user_id, action_code, entity_type, entity_id, entity_label y result_code. Para el historial de mascotas consulta pet_lifecycle_history, con los campos new_is_active, reason y created_at además del identificador de mascota. El repositorio no define el DDL canónico ni verifica tipos, claves o índices de Layerbase; confirmá esos metadatos antes de habilitar las operaciones.

Consulta manual de solo lectura para confirmar columnas e índices (sustituí el nombre de tabla si revisás también el historial de pacientes):

```
SELECT COLUMN_NAME, COLUMN_TYPE, IS_NULLABLE, COLUMN_DEFAULT, ORDINAL_POSITION
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'admin_sensitive_action_audit'
ORDER BY ORDINAL_POSITION;

SELECT INDEX_NAME, NON_UNIQUE, SEQ_IN_INDEX, COLUMN_NAME
FROM information_schema.STATISTICS
WHERE TABLE_SCHEMA = DATABASE() AND TABLE_NAME = 'admin_sensitive_action_audit'
ORDER BY INDEX_NAME, SEQ_IN_INDEX;
```

El backend necesita estas tablas para completar esas operaciones. Si la verificación detecta diferencias, detené la prueba funcional y consultá al responsable de la base. No realices cambios de esquema desde la aplicación ni en producción como parte de una prueba.

Contratos funcionales a validar en un entorno aislado con cuentas y registros ficticios: desactivación de usuario con permisos efectivos, invalidación de `session_version` y rechazo de login/refresh; rechazo de eliminación de roles del sistema y de roles personalizados asignados; rechazo de desactivación del último `SUPER_ADMIN` activo/verificado; registro de actor y acción. La verificación remota y una prueba end-to-end en Layerbase no forman parte de esta revisión documental.

La instalación de una base limpia no está cubierta por el contenido de este checkout: npm ci no crea el esquema y no se incluyen scripts SQL canónicos de bootstrap. Antes de aprovisionar otra base, obtené el paquete y el procedimiento aprobados por el responsable. El endpoint GET

`/api/cron/notifications` está registrado, pero `backend/vercel.json` está vacío y no se encontró programación local: la ejecución periódica depende de configuración externa autenticada. La API no implementa Expo Push ni Web Push; el cliente usa el centro interno de notificaciones.